Instituto Tecnológico de Costa Rica

Escuela de Ingeniería en Computadores

CE-1113 Sistemas Empotrados

Documento de Diseño (DI)

Proyecto I: Sistema embebido a la medida para un robot aspiradora autónomo con reproducción de audio y control remoto

Profesor

Dr.-Ing. Jeferson González Gómez

Integrantes

Gabriel Bolaños Barboza	2022327511
Yerik Chaves Serrano	2022437493
Emmanuel Esquivel Chavarría	2022312336
Sergio Marín Zuñiga	2018171984

6 de octubre de 2026

Índice

1. Introducción **2**

2. DI1: Identificación de necesidades y requerimientos **2**

3. DI2: Valoración de alternativas **3**

3.1. Aislamiento entre la lógica y los motores 4

3.2. Alimentación 4

3.3. Chasis 5

3.4. Generación del PWM de los motores 5

3.5. Sensores de proximidad 6

3.6. Indicadores visuales 6

3.7. Salida de audio 7

3.8. Servidor web 7

3.9. Conectividad 8

3.10. Otras decisiones 8

4. DI3: Diseño de la alternativa seleccionada **8**

4.1. Arquitectura de hardware 8

4.2. Protecciones del dominio de potencia 11

4.3. Arquitectura de software 11

5. DI4: Validación del diseño **12**

5.1. Validación funcional 12

5.2. Eficiencia de recursos 14

5.3. Seguridad, costo e impacto 16

5.4. Limitaciones y riesgos 16

6. Conclusiones **16**

1. Introducción

El proyecto consiste en el diseño de un sistema embebido a la medida que controla un robot aspiradora autónomo sobre una Raspberry Pi 4. El sistema navega evitando obstáculos, reproduce audio en formato MP3 y se opera de forma remota mediante una interfaz web por WiFi, todo sobre una imagen mínima de Linux construida con Yocto.

Este documento presenta las decisiones de diseño del grupo según los indicadores DI1 a DI4: la identificación de necesidades y requerimientos, la valoración de alternativas, el diseño de la alternativa seleccionada y su validación sobre el robot final (imagen `robot-demo1`, versión Demo 1).

2. DI1: Identificación de necesidades y requerimientos

Los requerimientos se obtuvieron de la especificación del Proyecto I y de las restricciones propias del equipo. Cada requerimiento técnico tiene un identificador y un criterio de aceptación verificable, que se usa en la validación de la Sección 5 (Tabla 1).

Tabla 1: Requerimientos técnicos con criterio de aceptación.

ID	Requerimiento	Criterio de aceptación
R1	Navegación autónoma con un algoritmo de cobertura reactiva.	Ante un obstáculo el robot se detiene, retrocede y cambia de dirección sin intervención.
R2	Al menos dos sensores de proximidad, con detección frontal y lateral, leídos en tiempo real.	Las lecturas se actualizan en la interfaz durante la operación.
R3	Tracción diferencial con dos motores DC y PWM por motor.	Avance, retroceso, giro a la izquierda y a la derecha, detención y radio de giro variable.
R4	Cuatro LEDs de estado.	Indican sistema encendido, modo autónomo, modo manual y obstáculo.
R5	Reproducción de MP3 local, concurrente con la navegación.	Desde la interfaz se elige canción, se reproduce, pausa, detiene y se ajusta el volumen.
R6	Notificaciones sonoras.	Suenan al iniciar el sistema, al iniciar el modo autónomo, al detectar un obstáculo y al pasar a modo manual.
R7	Panel de control remoto con mapa del recorrido.	Muestra modo activo, controles direccionales, sensores, audio, LEDs y una grilla 2D (visitada, obstáculo, desconocida) en tiempo real.
R8	Autenticación con un protocolo de seguridad mínimo.	Inicio de sesión obligatorio con al menos un usuario; credenciales cifradas en tránsito.
R9	Ejecución automática sin interfaz gráfica.	Servicio `systemd` propio con `Restart=on-failure`; `multi-user.target`.

ID	Requerimiento	Criterio de aceptación
R10	Biblioteca dinámica propia como único acceso al hardware.	El servidor usa solo la API de `librobot.so`.
R11	Imagen Yocto mínima, compilación cruzada con CMake y receta `.bb` propia.	La imagen se reproduce con un solo comando `bitbake robot-demo1`.
R12	Eficiencia de recursos.	Rootfs de 200 MB como máximo, arranque de 15 s como máximo; RAM y CPU medidos en operación.
R13	Seguridad eléctrica del equipo prestado.	Aislamiento óptico entre GPIO y potencia, tierras separadas, 5 V regulados para la Raspberry Pi y BMS en la batería.

La Tabla 2 resume las necesidades no técnicas que también condicionaron el diseño.

Tabla 2: Necesidades de seguridad, costo, ambientales, sociales y del equipo.

Aspecto	Necesidad y justificación
Seguridad de personas	Las celdas de ion de litio tienen riesgo térmico y de incendio, por lo que se protegen con BMS. Las ruedas y el motor de aspirado son partes móviles, por lo que el control remoto exige inicio de sesión para evitar que otra persona mueva el robot.
Costos	La Raspberry Pi 4 es un préstamo del curso. El resto del robot debía costar lo menos posible: el presupuesto final fue de aproximadamente 65 000 colones.
Impacto ambiental	Reducir residuos electrónicos y consumo energético: usar celdas recargables protegidas para prolongar su vida útil y fabricar solo las piezas mecánicas necesarias.
Sociales	Un robot de uso doméstico debe operarse desde cualquier dispositivo sin instalar software, y su ruido y movimiento no deben causar riesgo a personas o mascotas.
Recursos del equipo	Cuatro integrantes, fecha de entrega fija, un solo entorno de construcción Yocto en máquina virtual, y toda dependencia de software instalada mediante recetas (sin `pip` ni `npm`).

3. DI2: Valoración de alternativas

Para cada decisión principal se compararon alternativas que cumplen la necesidad. Se usaron como criterios el aspecto técnico (cumplimiento y complejidad), el costo, la seguridad y, cuando aplica, el aspecto ambiental. Cada alternativa se valoró como Alta, Media o Baja según qué tan favorable es en cada criterio.

3.1 Aislamiento entre la lógica y los motores

Tabla 3: Alternativas de aislamiento de la etapa de potencia.

Alternativa	Técnico	Costo	Seguridad	Observación
GPIO directo al L298N	Alta	Alta	Baja	Incumple la nota de seguridad obligatoria; un fallo puede destruir la tarjeta prestada.
Optoacopladores PC817 + L298N	Alta	Alta	Alta	Aísla las señales IN y EN; corriente de 5 a 10 mA compatible con los GPIO; bajo costo.
Controlador con aislamiento integrado (p. ej. DRV8871)	Alta	Media	Alta	Requiere verificar la etapa de entrada y adquirir módulos adicionales.

Decisión: optoacopladores PC817 con un puente H L298N, porque cumplen el requisito de aislamiento con los componentes de menor costo y disponibles de inmediato.

3.2 Alimentación

Tabla 4: Alternativas de fuente de alimentación portátil.

Alternativa	Técnico	Costo	Seguridad	Observación
Pack 1S/2S 18650 con elevador	Media	Alta	Alta	Tras la caída del L298N la tensión en los motores queda por debajo de su mínimo de 6 V; requiere un elevador de alta corriente.
Pack 3S 18650 con BMS	Alta	Media	Alta	De 9 V a 12.6 V, dentro del rango de 6 V a 12 V de los motores; se reduce a 5 V con convertidores reductores.
Batería LiPo de modelismo	Alta	Media	Media	Mayor riesgo ante perforación o sobrecarga; requiere cargador balanceador específico.

Decisión: pack 3S de celdas 18650 con un BMS 3S de 40 A, porque alimenta el L298N directamente dentro del rango de los motores y el BMS protege las celdas contra sobrecarga, sobredescarga y cortocircuito. Dos convertidores reductores generan rieles de 5 V independientes: uno solo para la Raspberry Pi (cerca de 3 A) y otro para el motor de aspiración y la lógica del L298N.

3.3 Chasis

Tabla 5: Alternativas de chasis.

Alternativa	Técnico	Costo	Ambiental	Observación
Reutilizar el chasis de una aspiradora robótica tipo Roomba	Baja	Alta	Alta	Reduce residuos, pero en la prueba de montaje no tuvo espacio para la Raspberry Pi, la batería, el L298N y los convertidores.
Chasis de acrílico cortado	Media	Media	Media	Requiere diseño y corte; las piezas planas dificultan una forma circular y la fijación de componentes en altura.
Chasis impreso en 3D de dos pisos	Alta	Media	Media	Diseñado a la medida de los componentes; los dos pisos dan espacio suficiente y permiten la forma circular.

Decisión: chasis impreso en 3D con dos pisos. La primera opción fue reutilizar el chasis de una aspiradora tipo Roomba, que favorecía el costo y el impacto ambiental, pero al montar los componentes no hubo espacio suficiente; además, su motor original solo movía la rueda loca y no permitía tracción diferencial. El chasis propio se diseñó con dos pisos para asegurar que todos los componentes calzaran.

3.4 Generación del PWM de los motores

Esta decisión surgió durante la integración: al mover los motores, la salida de audio del jack quedaba muda (`mpg123`: `Cannot flush to my output anymore`). En la Raspberry Pi el audio analógico se genera con el mismo periférico PWM que el PWM por hardware de los GPIO 18 y 19.

Tabla 6: Alternativas para el PWM de tracción.

Alternativa	Técnico	Costo	Audio	Observación
PWM por hardware (GPIO 18 y 19)	Alta	Alta	Baja	Señal más precisa, pero inutiliza el jack de 3.5 mm mientras los motores funcionan.
PWM por software con DMA de `pigpio`	Alta	Alta	Alta	1 kHz con resolución suficiente para motores DC; no usa el periférico de audio.
DAC externo por I2S y PWM por hardware	Media	Media	Alta	Requiere un módulo adicional; I2S usa los GPIO 18 a 21, ya ocupados por los EN de los motores.

Decisión: PWM por software con DMA de `pigpio` a 1 kHz, porque conserva la salida de audio sin hardware adicional y la precisión es suficiente para controlar la velocidad de motores DC.

3.5 Sensores de proximidad

Tabla 7: Alternativas de sensores laterales (el frontal es un HC-SR04).

Alternativa	Técnico	Costo	GPIO	Observación
HC-SR04 laterales	Media	Alta	Baja	Miden distancia, pero cada lectura bloquea hasta 35 ms y usa dos GPIO; tres sensores alargan el ciclo de lectura.
Infrarrojos TCRT5000L laterales	Alta	Alta	Alta	Salida digital de lectura inmediata y un solo GPIO; detectan presencia a corta distancia (0.2 a 15 mm).

Decisión: un HC-SR04 frontal, que mide la distancia en la dirección de avance, y dos TCRT5000L laterales, que detectan contacto cercano en los costados sin bloquear el ciclo de lectura.

3.6 Indicadores visuales

Tabla 8: Alternativas para los cuatro LEDs de estado.

Alternativa	Técnico	Costo	GPIO	Observación
Cuatro LEDs discretos con resistencia	Alta	Alta	Baja	Diseño inicial del esquemático; usa cuatro GPIO.
Tira de 4 LEDs SK6812MINI-E por SPI	Alta	Alta	Alta	Un solo GPIO (MOSI de SPI0) y un color distinto por indicador.

Decisión: tira de cuatro LEDs SK6812MINI-E controlada por SPI0 (GPIO10), porque libera GPIO y cada indicador tiene un color propio: blanco (sistema), verde (autónomo), azul (manual) y rojo (obstáculo).

3.7 Salida de audio

Tabla 9: Alternativas de salida de audio.

Alternativa	Técnico	Costo	Seguridad	Observación
Jack 3.5 mm + amplificador PAM8403	Alta	Alta	Alta	No consume GPIO; usa el driver `snd-bcm2835` del kernel.
DAC externo por I2S	Baja	Media	Alta	Conflicto de pines con los EN de los motores (GPIO 18 y 19).

Decisión: jack de 3.5 mm con un módulo amplificador PAM8403, porque no requiere módulos adicionales ni GPIO. Su condición es no usar el PWM por hardware (subsección anterior).

3.8 Servidor web

Tabla 10: Alternativas para el servidor de control remoto.

Alternativa	Técnico	Recursos	Integración	Observación
Python (p. ej. Flask)	Media	Baja	Media	Requiere intérprete y módulos en la imagen; dependencias habituales por `pip`.
Node.js	Media	Baja	Baja	Agrega un entorno de ejecución grande al rootfs; dependencias por `npm`.
C con Mongoose	Alta	Alta	Alta	Un par de archivos fuente; enlaza directamente con `librobot.so`; se compila con CMake dentro de Yocto.

Decisión: servidor en C con Mongoose 7.23, porque mantiene el rootfs pequeño, no requiere dependencias fuera de Yocto y llama a la biblioteca sin capas intermedias. La interfaz es una aplicación web en lugar de una aplicación móvil porque funciona en cualquier dispositivo de la red sin instalación (Tabla 2, aspecto social).

3.9 Conectividad

Tabla 11: Alternativas de direccionamiento en la red WiFi.

Alternativa	Técnico	Uso	Observación
Solo DHCP	Alta	Baja	La dirección cambia y para conocerla hace falta un monitor conectado al robot.
IP fija + DHCP	Alta	Alta	Dirección conocida (172.20.10.14 en el hotspot) aunque el robot esté en movimiento, y DHCP como respaldo.

Decisión: `wpa_supplicant` con `systemd-networkd`, con una IP fija y DHCP a la vez, porque durante la operación no hay acceso físico a la consola del robot.

3.10 Otras decisiones

- **Reproductor MP3:** `mpg123`, disponible como receta en Yocto. Su modo de control remoto (`-R`) permite pausar, reanudar y ajustar el volumen sin reiniciar la reproducción.
- **Acceso al GPIO:** biblioteca `pigpio`, que ofrece PWM por software con DMA y medición de pulsos con resolución de microsegundos para el HC-SR04.

4. DI3: Diseño de la alternativa seleccionada

4.1 Arquitectura de hardware

La Figura 1 muestra el esquemático en KiCad. Los convertidores aparecen con el símbolo genérico GL2576; en el montaje se usan los módulos reductores definidos en ADR-002. El esquemático conserva dos elementos del diseño inicial que se cambiaron en el montaje: los LEDs discretos, reemplazados por la tira SK6812, y el amplificador LM386, reemplazado por un módulo PAM8403.

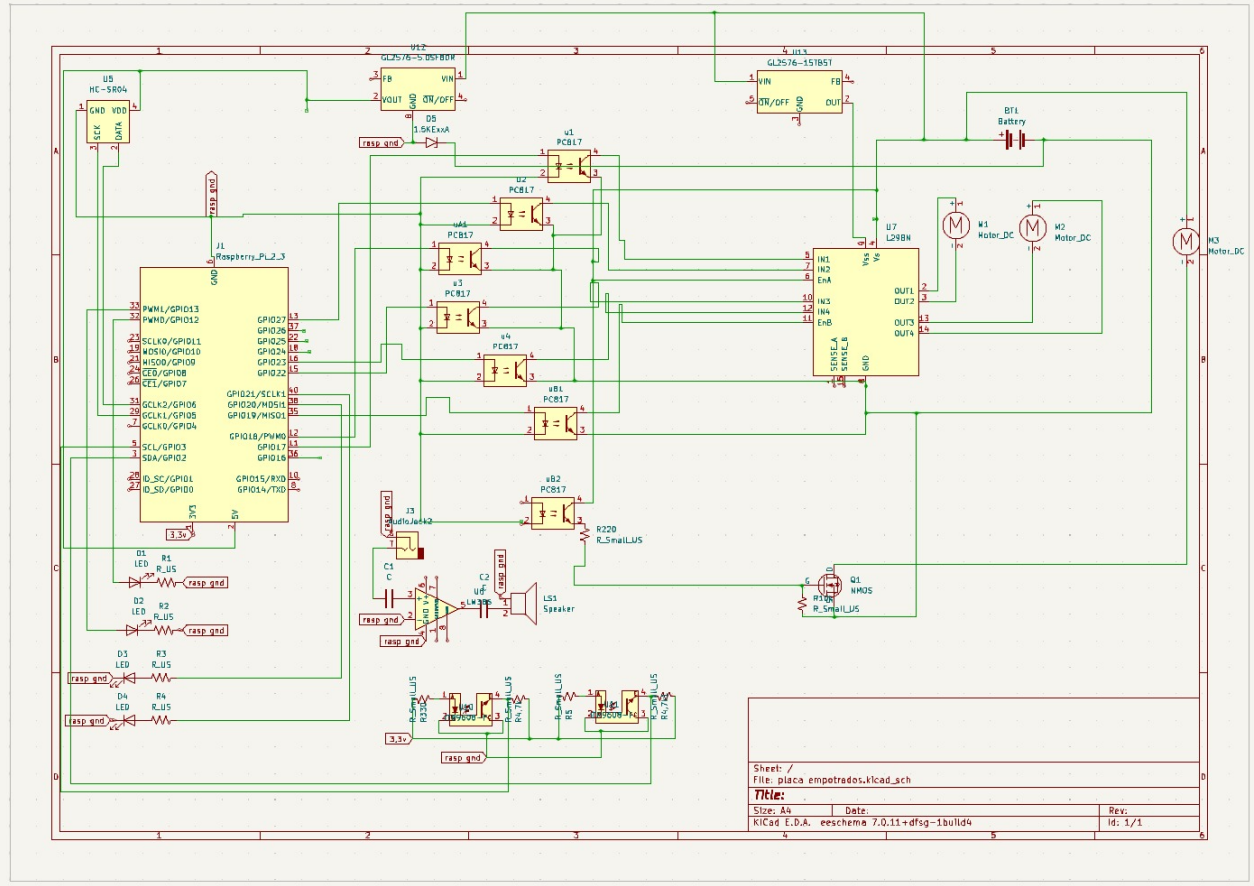


Figura 1: Esquemático del circuito de control (KiCad).

El diseño deriva tres ramas de la salida protegida del BMS (Figura 2):

- **VS del L298N:** tensión directa del pack para los dos motores de tracción.
- **Reductor #1 (5.0 a 5.1 V):** Raspberry Pi y periféricos lógicos (sensores, LEDs y amplificador).
- **Reductor #2 (5 V):** motor de aspiración y lógica del L298N. Se retira el jumper 5V-EN del módulo porque su regulador interno no debe recibir 12.6 V.

Los reductores no son aislados y quedan referidos al negativo del pack. Por eso, según la solución acordada con el profesor, se coloca un diodo en la tierra de entrada del reductor #1 que bloquea las corrientes y picos de retorno del dominio de potencia hacia el dominio lógico y la batería. Las señales entre ambos dominios pasan por optoacopladores PC817.

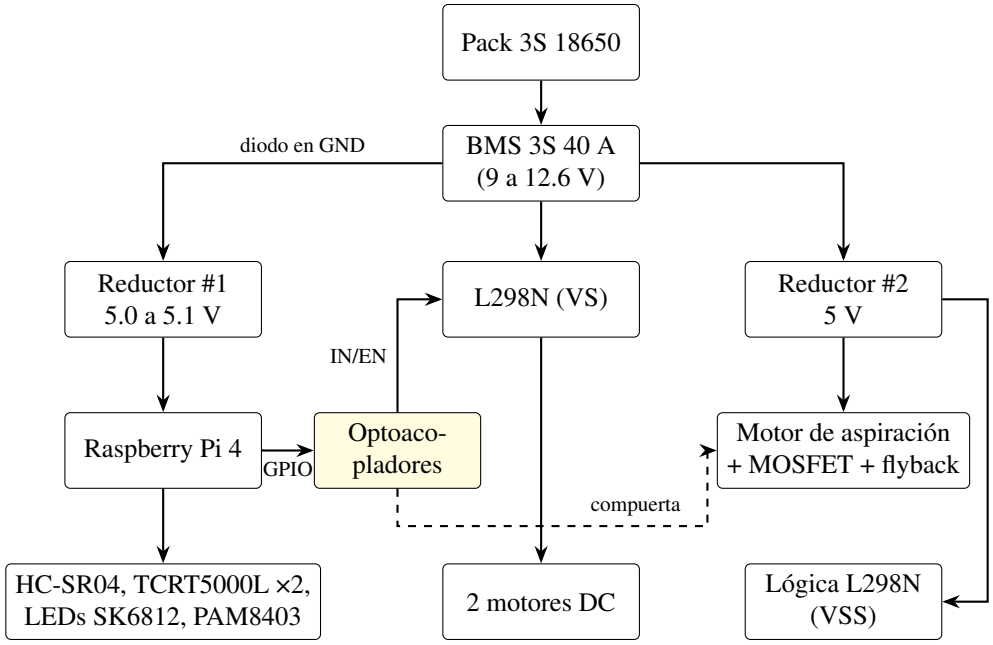


Figura 2: Distribución de alimentación y control (según ADR-002).

La Tabla 12 muestra la asignación final de pines (numeración BCM), tal como quedó en `robot_pins.h` después de las pruebas en el robot.

Tabla 12: Asignación final de GPIO.

Función	**GPIO**	**Observación**
Motor izquierdo IN1 / IN2 / EN	22 / 23 / 19	Canal B del L298N; PWM por software a 1 kHz
Motor derecho IN1 / IN2 / EN	17 / 27 / 18	Canal A del L298N; PWM por software a 1 kHz
Motor de aspirado	26	Optoacoplador + MOSFET; encendido con nivel alto
HC-SR04 frontal TRIG / ECHO	16 / 4	ECHO con divisor 1 kΩ / 2 kΩ
TCRT5000L izquierdo / derecho	5 / 25	Salida digital, activa en bajo
LEDs de estado (SK6812 ×4)	10 (SPI0 MOSI)	330 Ω en serie; alimentados a 3.3 V
Audio	Jack 3.5 mm	Amplificador PAM8403; sin uso de GPIO

- **Intercambio de canales:** en la primera prueba de tracción el canal A del L298N movía la rueda derecha. Se intercambiaron los bloques de pines en `robot_pins.h` en lugar de recablear, porque el cambio es de una línea y no modifica el esquemático.
- **Polaridad invertida por los optoacopladores:** el PC817 invierte la señal hacia el L298N. Por eso la biblioteca escribe el ciclo de trabajo como $100 - v$ para una velocidad v en porcentaje, y un PWM de 0 en el GPIO equivale a velocidad máxima.
- **Acondicionamiento de señales:** el ECHO del HC-SR04 entrega 5 V, por lo que pasa por un divisor que

lo reduce a 3.33 V. El TRIG no requiere adaptación, porque el sensor reconoce 3.3 V como nivel alto.

4.2 Protecciones del dominio de potencia

- **Motor de aspiración:** gira en un solo sentido, por lo que se controla con un MOSFET canal N de lado bajo y no con un puente H. La compuerta se maneja desde un optoacoplador con una resistencia serie de 220 Ω y tiene una resistencia de 10 kΩ hacia la fuente (source), para que el motor quede apagado sin señal.
- **Diodo flyback:** en paralelo con el motor de aspiración, para limitar el pico de tensión al apagar el MOSFET. Los motores de tracción usan los diodos de protección del módulo L298N, porque su polaridad se invierte con el sentido de giro.
- **Ruido y desacoplo:** capacitor cerámico de 100 nF entre los terminales de cada motor, y capacitores cerca del L298N y del circuito de aspiración para absorber los picos de arranque.
- **Ajuste de los reductores:** la salida de cada convertidor se ajusta y se mide con multímetro antes de conectar cualquier carga.

4.3 Arquitectura de software

El software se organiza en capas (Figura 3). Ninguna capa superior accede directamente al hardware: el servidor solo usa la API de `librobot.so`.

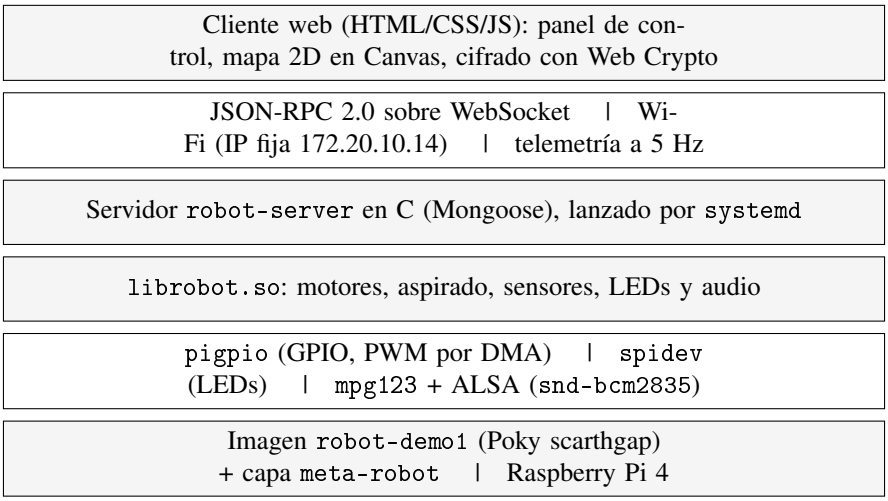


Figura 3: Arquitectura de software por capas.

- **Capa `meta-robot`:** contiene las recetas propias (`librobot`, `pigpio`, `robot-server`, `robot-media`, `robot-wifi`, `robot-startup-sound`, `robot-metricas`), los `.bbappend` de `mpg123` y del arranque de la Raspberry Pi, y la receta de imagen `robot-demo1`. La imagen completa se reproduce con `bitbake robot-demo1`, porque la receta de imagen declara todos los paquetes del proyecto; `local.conf` solo fija la máquina (`raspberrypi4`), el bus SPI y los parámetros de arranque de la Raspberry Pi.
- **Biblioteca `librobot.so`:** cada módulo (motor, sensores, LEDs, audio) tiene su propia inicialización y limpieza, coordinadas por `robot_init()` y `robot_cleanup()`. Los cambios de velocidad se aplican en rampa (5 % cada 15 ms) y un cambio de sentido pasa primero por velocidad cero, para limitar los picos de corriente en el L298N.

- **Servidor:** bucle de eventos de un solo hilo de Mongoose con un temporizador de 200 ms que ejecuta la navegación autónoma, actualiza el mapa y envía la telemetría a los clientes autenticados. Las lecturas de sensores se hacen en un hilo aparte, porque el HC-SR04 bloquea hasta 35 ms. Al recibir `SIGTERM` detiene motores y aspiradora y libera el hardware.
- **Navegación autónoma:** algoritmo aleatorio con rebote. El robot avanza hasta que un sensor detecta obstáculo (frontal a 18 cm o menos, o cualquiera lateral); entonces se detiene, retrocede 0.8 s y gira hacia el lado libre.
- **Mapa 2D:** grilla de 25 × 25 celdas de 15 cm (visitada, obstáculo o desconocida) que se actualiza con la odometría estimada por tiempo y las lecturas de los sensores. Solo se envían las celdas que cambiaron.
- **Seguridad mínima:** las credenciales viajan cifradas con AES-256-CBC (Web Crypto en el cliente y OpenSSL EVP en el servidor) y se almacenan en `/var/lib/robot-server/users.db`.
- **Arranque:** `systemd` levanta los servicios en `multi-user.target`: sonido de inicio, WiFi, servidor (con `Restart=on-failure`), un aviso con la IP en pantalla y la medición de arranque de `robot-metricas`.

5. DI4: Validación del diseño

La validación se hizo sobre el robot armado con la imagen `robot-demo1` (Figura 4).



(a) Modelo físico con el HC-SR04 frontal.



(b) Robot en operación con los LEDs de sistema y modo manual encendidos.

Figura 4: Robot aspiradora final.

5.1 Validación funcional

Tabla 13: Validación de los requerimientos técnicos.

ID	Evidencia	Estado
R1	En modo autónomo el robot esquiva obstáculos: se detiene, retrocede y gira.	Cumple
R2	El panel muestra en tiempo real la distancia frontal (48.8 cm en la Figura 5) y el estado de los dos laterales.	Cumple
R3	El control manual mueve el robot en las cuatro direcciones, detiene y ajusta velocidad y radio de giro.	Cumple, con un retardo leve al cambiar de dirección
R4	Los cuatro LEDs indican correctamente: blanco (sistema), verde (autónomo), azul (manual) y rojo (obstáculo).	Cumple
R5	Desde la interfaz se elige canción y funcionan reproducir, pausar, reanudar, detener y volumen, mientras el robot navega.	Cumple
R6	Suenan las cuatro notificaciones. La de inicio se escucha más baja porque se superpone con el motor de aspiración.	Cumple
R7	El mapa marca en verde las celdas recorridas y en rojo los obstáculos, en tiempo real (Figura 7).	Cumple
R8	El inicio de sesión funciona, permite registrar cuentas nuevas y acceder con ellas; credenciales cifradas con AES-256 en tránsito (Figura 6).	Cumple
R9	`robot-server` queda `active` al arrancar, con `Restart=on-failure` y sin interfaz gráfica.	Cumple
R10	El servidor enlaza solo con `librobot.so` para el hardware.	Cumple
R11	La imagen se construye con `bitbake robot-demo1`; la compilación cruzada de `librobot` y `robot-server` usa CMake y la toolchain `arm-poky-linux-gnueabi`.	Cumple
R12	Ver Tabla 14.	Cumple
R13	Señales de control por optoacopladores y diodo en la tierra del reductor de la Raspberry Pi; BMS en el pack.	Cumple

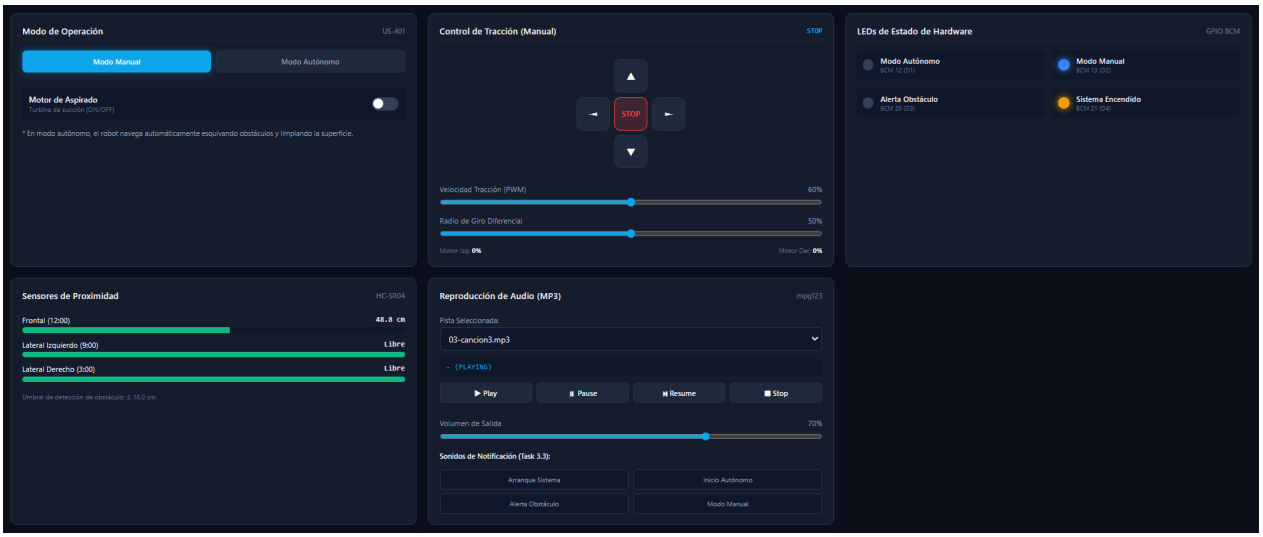


Figura 5: Panel de control: modo, aspirado, tracción manual, LEDs, sensores y audio.

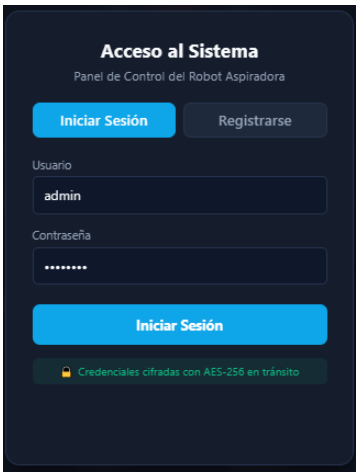


Figura 6: Inicio de sesión.

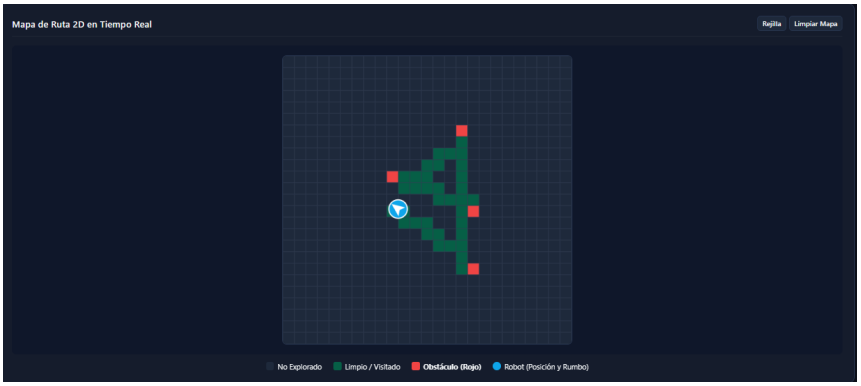


Figura 7: Mapa 2D: recorrido (verde) y obstáculos (rojo).

5.2 Eficiencia de recursos

Las métricas se tomaron en la Raspberry Pi con el script `robot-metricas` de la imagen, que usa `du`, `df`, `systemctl`, `systemd-analyze`, `/proc/meminfo`, `/proc/stat` y `/proc/<pid>/stat`. La RAM y la CPU se midieron en operación normal, como pide la especificación: robot en modo autónomo, reproducción de música y servidor web activos. La CPU se promedió durante 30 s, a los 54 s del arranque.

Tabla 14: Métricas de la imagen final.

Métrica	Resultado	Referencia	Método
Tamaño del rootfs	68 MB (partición: 76 de 141 MB)	200 MB	`du -sxm /` y `df -m /`
Arranque hasta servidor operativo	5.37 s desde el inicio del kernel	15 s	`ActiveEnterTimestampMonotonic` de `robot-server`
RAM en uso	100 MB de 1868 MB (5.3 %)	–	`/proc/meminfo`
RAM por proceso	`robot-server` 3.8 MB; `mpg123` 2.4 MB	–	`VmRSS` en `/proc/<pid>/status`
CPU total (4 núcleos)	2.4 %	–	`/proc/stat`, 30 s
CPU por proceso	`robot-server` 9.6 %; `mpg123` 0.9 % (de un núcleo)	–	`/proc/<pid>/stat`, 30 s
Temperatura del SoC	45.8 °C	–	`thermal_zone0`

El rootfs ocupa la tercera parte del presupuesto y el servidor queda operativo en un tercio del tiempo de referencia. La medición de arranque no incluye la etapa de firmware de la Raspberry Pi, anterior al kernel; el margen de 9.6 s respecto al presupuesto permite absorberla.

La Tabla 15 justifica los paquetes agregados a `core-image-minimal`.

Tabla 15: Paquetes agregados a la imagen mínima.

Paquete	Justificación
`librobot0`, `librobot-selftest`	Biblioteca de control (R10) y programa de prueba del hardware.
`pigpio`	GPIO, PWM por DMA y medición de pulsos del HC-SR04.
`kernel-module-spi-bcm2835`, `kernel-module-spidev`	Bus SPI para la tira de LEDs.
`mpg123`, `alsa-utils`	Decodificador MP3 y mezclador del jack (R5, R6).
`robot-startup-sound`, `robot-media`	Sonido de inicio, canciones y notificaciones (R6).
`robot-server` (con OpenSSL)	Servidor web y cifrado de credenciales (R7, R8, R9).
`robot-wifi`, `wpa-supplicant`, `iw`	Conexión WiFi con IP fija.
`kernel-module-brcmfmac`, `kernel-module-brcmfmac-wcc`, `linux-firmware-rpidistro-bcm43455`, `wireless-regdb-static`	Driver, módulo de proveedor y firmware del WiFi de la Raspberry Pi 4.
`robot-metricas`, `systemd-analyze`	Medición de las métricas obligatorias (R12).

5.3 Seguridad, costo e impacto

- **Seguridad:** ningún GPIO se conecta directamente a la etapa de potencia; el control remoto exige inicio de sesión; el BMS protege las celdas.
- **Costo:** aproximadamente 65 000 colones sin contar la Raspberry Pi 4 prestada.
- **Impacto ambiental:** el chasis se imprimió a la medida, por lo que solo se fabricaron las piezas necesarias; las celdas son recargables y deben desecharse en un centro de acopio de baterías al final de su vida útil.

5.4 Limitaciones y riesgos

- **Retardo al cambiar de dirección:** la biblioteca aplica la rampa de velocidad primero a un motor y después al otro, por lo que al arrancar o invertir el sentido un motor empieza hasta unos 180 ms antes (a 60 %). La mejora propuesta es aplicar la rampa a ambos motores en el mismo ciclo.
- **Lógica de la aspiradora:** las pruebas A/B en el robot mostraron que el motor de aspiración enciende con GPIO26 en alto. Esa lógica quedó resuelta en el servidor; falta unificarla en `librobot` para que `vacuum_on()` corresponda al nivel alto.
- **Odometría estimada por tiempo:** sin encoders, el mapa acumula error con la distancia recorrida.
- **Sensores laterales:** el TCRT5000L detecta solo a pocos milímetros y su umbral depende del potenciómetro del módulo y del color de la superficie.
- **Seguridad de la red:** el canal no usa TLS y la clave simétrica se comparte entre cliente y servidor; corresponde al protocolo mínimo que pide la especificación. Las credenciales por defecto (`admin`) deben cambiarse antes de una demostración pública.

6. Conclusiones

El diseño cumple los requerimientos de la especificación sobre el robot físico: navegación autónoma con evasión de obstáculos, mapa en tiempo real, audio concurrente con notificaciones, control remoto autenticado y arranque automático. La imagen usa 68 MB de rootfs y el servidor queda operativo a los 5.37 s, ambos dentro del presupuesto.

Las decisiones priorizaron la seguridad del equipo prestado (aislamiento óptico, dominios de alimentación separados y BMS) y el bajo costo. La integración obligó a cambiar tres decisiones iniciales con evidencia: el chasis reutilizado se reemplazó por uno impreso en 3D de dos pisos por falta de espacio, el PWM pasó de hardware a software para no perder el audio, y los sensores laterales y los LEDs se cambiaron por componentes que usan menos GPIO.

Referencias

- González Gómez, J. (2026). Especificación Proyecto I: Sistema embebido a la medida para un robot aspiradora autónomo. Instituto Tecnológico de Costa Rica.
- Yocto Project. Yocto Project Documentation (scarthgap). `https://docs.yoctoproject.org`
- Raspberry Pi BSP layer. meta-raspberrypi. `https://github.com/agherzan/meta-raspberrypi`
- joan2937. pigpio library. `https://abyz.me.uk/rpi/pigpio/`
- Cesanta. Mongoose Networking Library. `https://mongoose.ws`